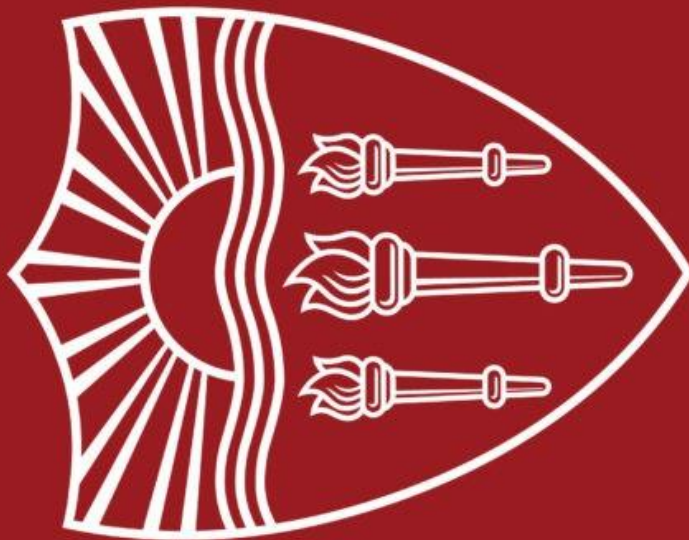




Lecture 2: n-gram Language Models

Instructor: Xiang REN
USC CSCI 544 Applied NLP
Fall 2026

Announcements

Logistics and Announcements

Always check the website and pay attention to Brightspace Announcement

Class Project

- There are some posts looking for teams, check those posts if you need !
- Mandatory 6 members/team
- Can be mixed among DEN & in-person students. But each one needs to contribute to the project!

Course video recordings are available on Brightspace

CARC access - We are working on it!

Interest in research in my lab

Logistics and Announcements

Class Enrollment

- CS department handles the enrollment, I have no control or any influence on it
- I have not knowledge if there's a “waitlist” of the class..
- Please do not email me about enrollment

Class Participation

You are required to attend class:

- We use Quiz for in-class attendance checks
 - We check Piazza activities for engagement in course discussion
 - In case of extenuating circumstances that prevent you from attending, use the Google Form
-
- For DEN students, live attendance is not required; but we still have Quiz to check on the understanding of course materials
 - Piazza engagement also count towards participation

Class Absence Form

Link in: **Activities > Assignments**

- Attendance part of participation grade
- Submit 24h before class start
- Prior instructor approval for non-medical/academic reasons
- Attach evidence file
 - Note from doctor/academic advisor
 - Conference registration confirmation
 - Explicit approval from Xiang

Date of missed lecture *

Date

mm/dd/yyyy 

Reason for absence (other reasons must be approved by instructor) *

☐ medical

☐ academic travel/appointment

☐ Other: _____

Evidence of reason (required)

examples of accepted proof include:

- note from doctor
- note from academic advisor
- approval from course instructor
- conference registration confirmation

Upload up to 5 supported files. Max 10 MB per file.

 [Add file](#)

Communication

We will give extra credits who helps the class a lot!

- Enroll in Piazza
- All course-related questions should be asked in Piazza. Please make it **public** (when appropriate) so that others can also help on the topics.
- There is “private” post in Piazza, only use this if it is about personal situation. Do not use it if it is about general class materials, homework, projects, etc.
- Do not email the instructor directly

Class Project

- Styled like a research or demo project
- Teams of 6 students
 - We expect to see 45 groups, based on enrollment
- **Project team formation due by Sep 3.**

Pay attention to the website update!

https://inklab.usc.edu/cs544_f26/details/project/

Project Team Formation (due Sep 3)

On Brightspace:

- Navigate to **Course Tools → Groups → View Available Groups**.
- Coordinate with your teammates to join the same empty group (showing **0/6**).

Groups

View My Groups

Available Groups

Join an available group from each category listed.

Groups	Description	Members	Actions
Sample Group			
Group 1		0/4	Join Group
Group 2		0/4	Join Group
Group 3		0/4	Join Group

(detailed instructions [here](#))

Recap

What is

Natural Language

Processing ?

- Field at the intersection of computer science, AI (especially machine learning) and linguistics
- Goal: for computers to process human language, similar to human understanding, towards performing useful tasks
- Challenge: understanding and representing the meaning of language is something even humans struggle with
- Processing: produce outputs (Y) with language or text as input (X)

A General Framework for NLP Systems

- Create a function to map an **input X** into an **output Y** , where X and/or Y involve language.

Input X

Output Y

Task

Text

Continuing Text

Language Modeling

Text

Text in Other Language

Translation

Text

Label

Text Classification

Text

Linguistic Structure

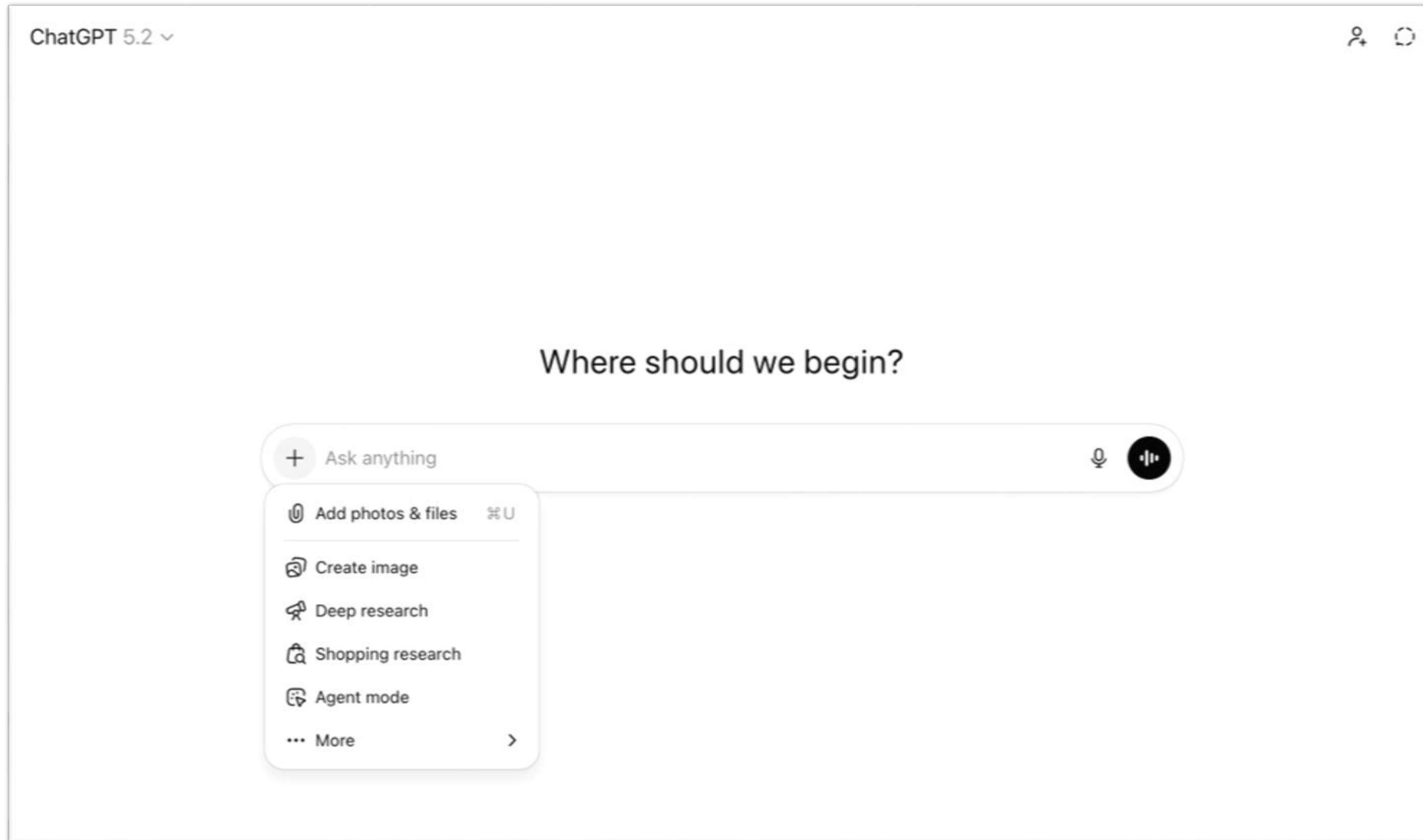
Language Analysis

Image

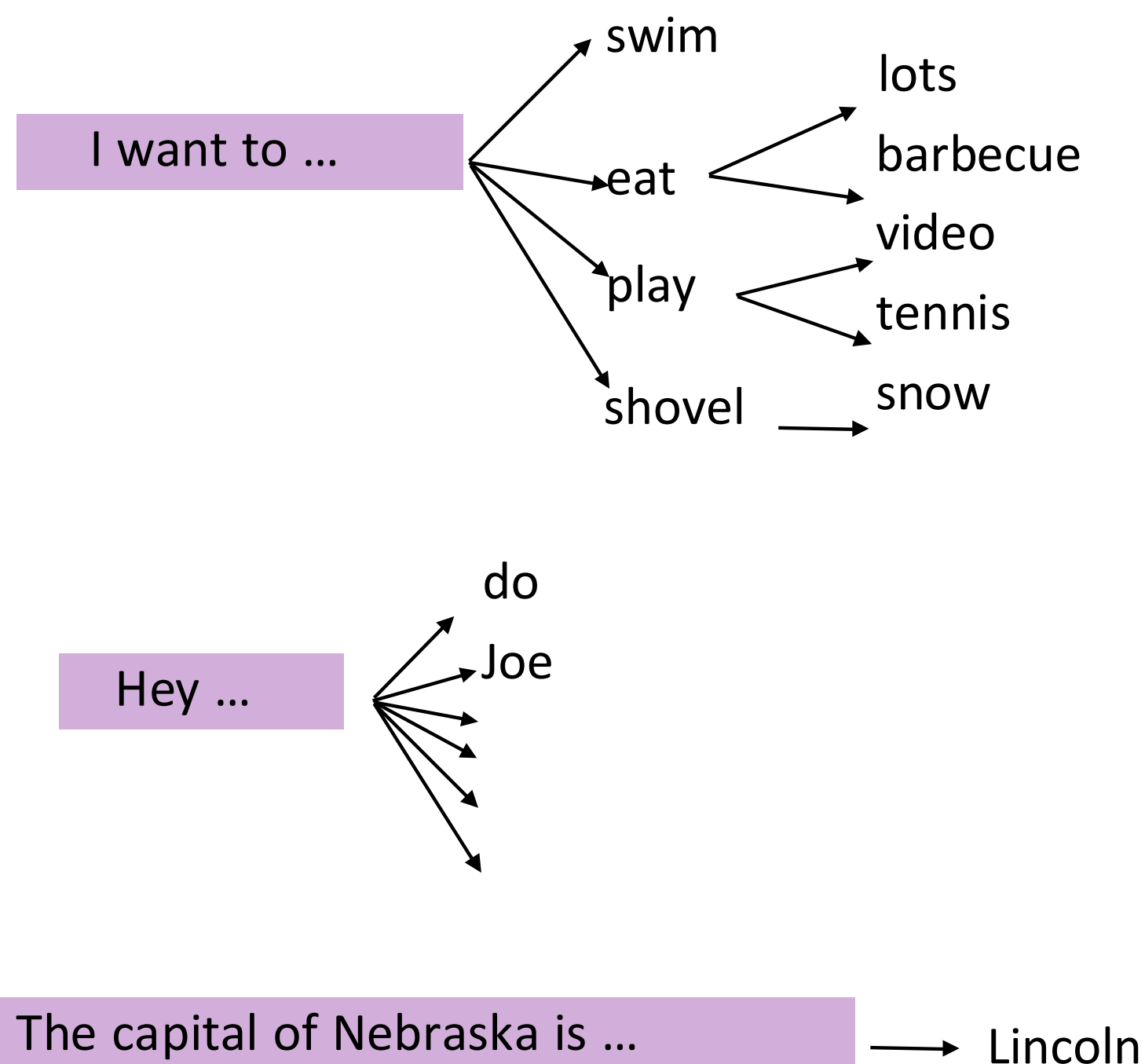
Text

Image Captioning

Today's NLP



Building a Language Model



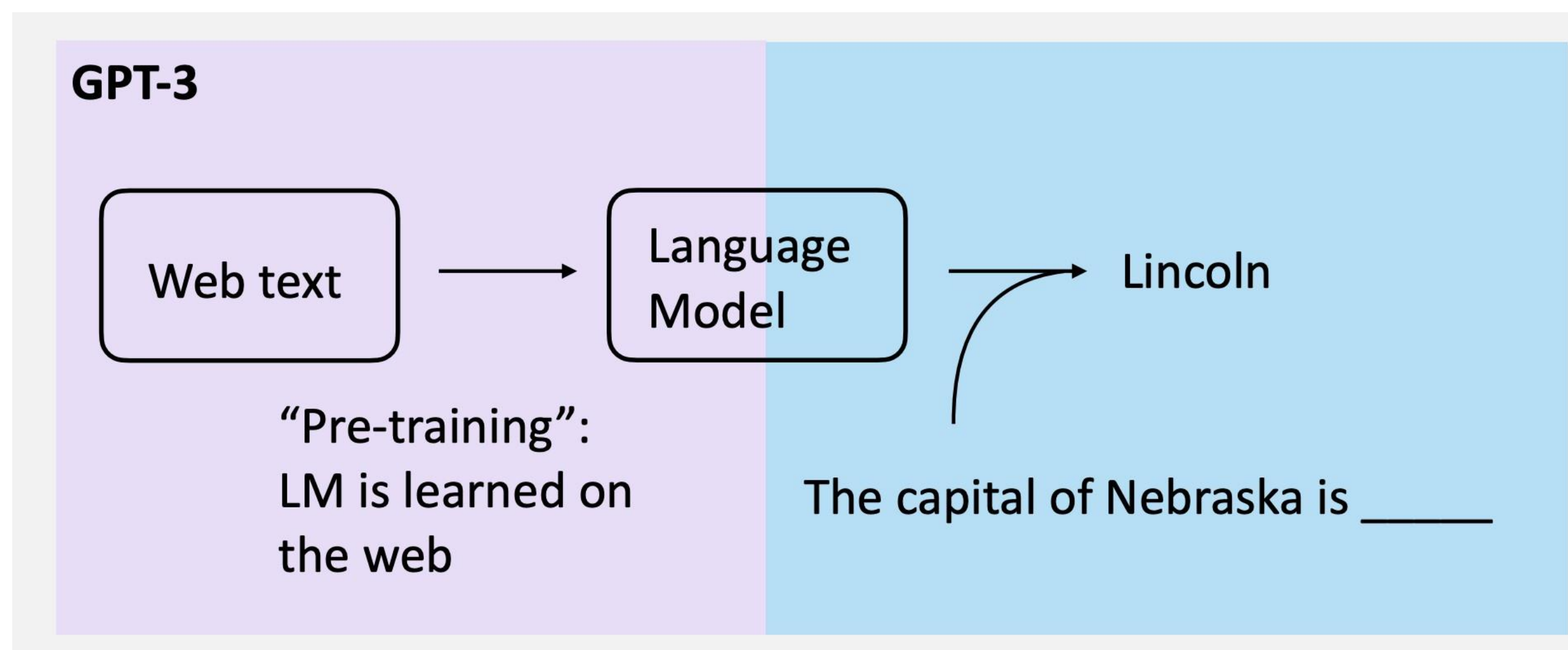
- Task: Given a sequence of words so far (the context), predict what comes next
- We never know for sure what comes next, but we can still make good guesses!

Certain sentence constructions are more likely than others, due to grammaticality, obscurity or commonness

Sentences have different probabilities!

Large Language Models (LLMs)

- Models are trained tons (terabytes) of data — a curated scrape of the web, and even generated data
- Have seen information about common and even not-so-common entities
- A big enough model can answer questions even without being trained to do so. What else can we get these models to do?



LLMs are being used to...

Summarize articles, podcasts or presentations

Code computer programs!

Script social media posts

Create a title for an article

Draft emails

Play games

Assist with job searches,
including writing resumes
and cover letters

Ask trivia questions

Compose music!!!

Discover keywords for search
engine optimization.

Formulate product
descriptions

Describe complex topics more simply

Solve math problems

Create articles, blog posts
and quizzes for websites

Reword existing content for a different medium, such
as a presentation transcript for a blog post

LLMs are being used to...

Summarize articles, podcasts or presentations

Code computer programs!

Script social media posts

Create a title for an article

Draft emails

Play games

Assist with job searches
including
and c

Ask trivia questions

**Extremely powerful: can in many cases replace
laborious manual efforts**

Describe complex topics more simply

engine optimization.

Solve math problems

Formulate product
descriptions

Create articles, blog posts
and quizzes for websites

Reword existing content for a different medium, such
as a presentation transcript for a blog post

The Pre-Neural Age of LMs (-2013)

- n-gram language models
- The power of context
- Smoothing
- Evaluating LMs

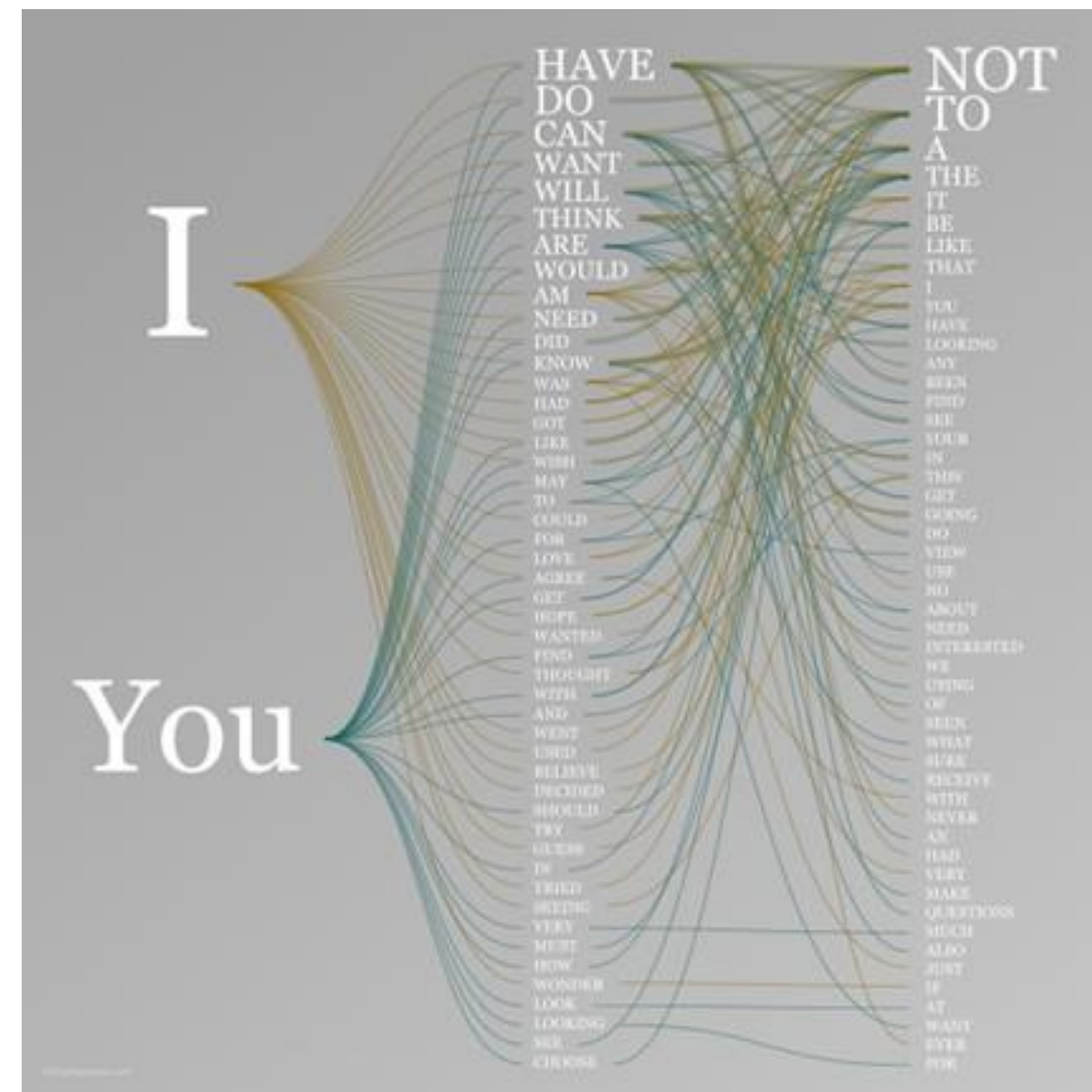


Image Courtesy: Chris Harrison

Lecture Outline

1. Announcements + Recap 
2. Probabilistic Language Models
3. n-gram Language Models
4. Evaluation and Perplexity
5. Generating from an n-gram Language Model
 - i. Zeroes
6. Smoothing

Probabilistic Language Models!

Assign a probability to a sentence

Probabilistic Language Modeling

Goal: compute the probability of a sentence or sequence of words:

$$P(\mathbf{w}) = P(w_1, w_2, \dots, w_n)$$



Difference

Related task: probability of an upcoming word: $P(w_n | w_1, w_2, \dots, w_{n-1})$

A model that assigns probabilities to sequences of words (e.g., either of these: $P(\mathbf{w})$ or $P(w_n | w_1, w_2, \dots, w_{n-1})$) is called a language model

How to compute $P(W)$?

“its water is so transparent that you can see the bottom”



$P(\text{its water is so transparent that you can see the bottom})$

$P(\text{its, water, is, so, transparent, that, you, can, see, the, bottom})$

How to compute this joint probability, $P(\mathbf{w}) = P(w_1, w_2, w_3, w_4, w_5, \dots, w_n)$?

e.g. $P(\text{its, water, is, so, transparent, that})$

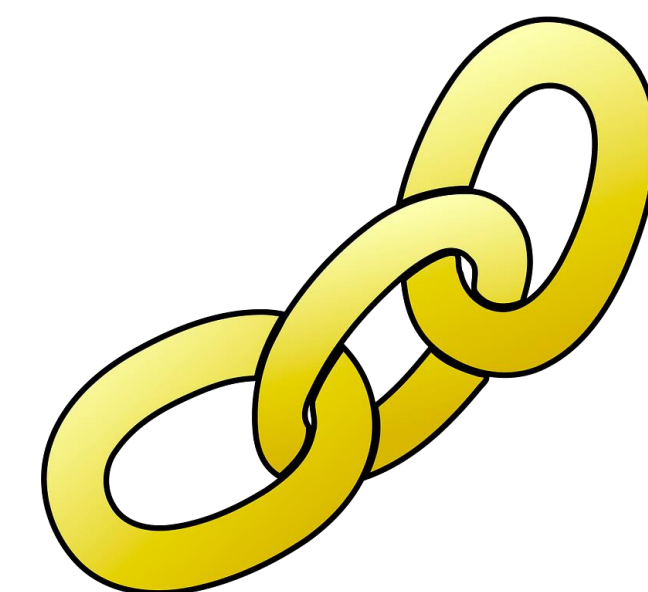
Intuition: let's rely on the Chain Rule of Probability

Chain Rule for words in a sentence

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_{i-1}, \dots, w_1)$$

$$\begin{aligned} P(\text{its water is so transparent}) = & P(\text{its}) \times \\ & P(\text{water} | \text{its}) \times \\ & P(\text{is} | \text{its water}) \times \\ & P(\text{so} | \text{its water is}) \times \\ & P(\text{transparent} | \text{its water is so}) \end{aligned}$$

Ordering matters in
language!



Why Probabilistic Models?

Why would you want to predict upcoming words, or assign probabilities to sentences?

- Probabilities are essential for language generation
- Any task in which we have to identify words in noisy, ambiguous input, like speech recognition
- For writing tools like spelling correction or grammatical error correction

I will be back soonish

I will be bassoon dish

Your so silly

You're so silly

Everything has improve

Everything has improved

Probabilistic Language Models

Writing assistant:

- $P(\text{high winds tonight}) > P(\text{large winds tonight})$

Spell Correction

- $P(\text{I'm about fifteen minuets away}) < P(\text{I'm about fifteen minutes away})$

Speech Recognition

- $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$

Summarization, question-answering, etc., etc. !!

But how to learn these probabilities?

Probability Estimation via Statistical Modeling



Suppose we have a biased coin that's heads with probability p

Suppose we flip the coin four times and see (H, H, H, T). What is p ?

We don't know what p is — could be 0.5! But $p = 3/4 = 0.75$ maximizes the probability of the single data sequence (H,H,H,T)

maximum likelihood estimate

The probability of the data is $p^3(1 - p)$: if you take the derivative and set it equal to zero and find $p = 0.75$

n-gram Language Model

The decision for what words occur after a word w is exactly the same as the biased coin, but with *many* possible outcomes (as many as all the words) instead of 2

Vocabulary

I like to **eat** cake but I want to **eat** pizza right now. Mary told her brother to **eat** pizza too.

$P(\text{next word} = \text{pizza} \mid \text{previous word} = \text{eat}) = 2/3$

$P(\text{next word} = \text{cake} \mid \text{previous word} = \text{eat}) = 1/3$

All other next words = 0 probability

$$P(w \mid w_{\text{prev}}) = \frac{\text{count}(w_{\text{prev}}, w)}{\text{count}(w_{\text{prev}})}$$

how many times do you see w_{prev} followed by w ?

how many times do you see w_{prev}

How to estimate the probability of the next word?

$$P(\text{that} \mid \text{its water is so transparent}) = \frac{\text{Count}(\text{its water is so transparent that})}{\text{Count}(\text{its water is so transparent})}$$

Could we just count and divide?

No! Too many possible sentences!

We'll never see enough data for estimating these

Markov Assumption

Simplifying Assumption:

$$P(\text{that} \mid \text{its water is so transparent}) \approx P(\text{that} \mid \text{transparent})$$

Or maybe...

$$P(\text{that} \mid \text{its water is so transparent}) \approx P(\text{that} \mid \text{so transparent})$$



Andrei Markov

Markov Assumption contd.

$$P(w_1, w_2, \dots, w_n) = \prod_i P(w_i | w_{i-k}, \dots, w_{i-1})$$

In other words, we approximate each component in the product such that it is only conditioned on the previous k elements

$$P(w_i | w_1, w_2, \dots, w_{i-1}) \approx P(w_i | w_{i-k} \dots w_{i-1})$$

k -th order Markov Assumption

Easiest case: first-order Markov Assumption

Mini Recap: Probabilistic Modeling

- What is a probabilistic language model?
- Why would we need one?
- How do we estimate one?
- How do we simplify the estimation problem?
- Next: a simple probabilistic language model



Lecture Outline

1. Announcements + Recap 
2. Probabilistic Language Models 
3. n-gram Language Models
4. Evaluation and Perplexity
5. Generating from an n-gram Language Model
 - i. Zeroes
6. Smoothing

n-gram Language Models

Simplest probabilistic model

Simplest Case: Unigram model

$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_i)$$

Some automatically generated sentences from a unigram model

- fifth, an, of, futures, the, an, incorporated, a, a, the, inflation, most, dollars, quarter, in, is, mass
- thrift, did, eighty, said, hard, 'm, july, bullish
- that, or, limited, the

Bigram Model

Condition on the previous word:

$$P(w_i | w_1, w_2, \dots, w_{i-1}) \approx P(w_i | w_{i-1})$$

Some automatically generated sentences from a bigram model

- texaco, rose, one, in, this, issue, is, pursuing, growth, in, a, boiler, house, said, mr., gurria, mexico, 's, motion, control, proposal, without, permission, from, five, hundred, fifty, five, yen
- outside, new, car, parking, lot, of, the, agreement, reached
- this, would, be, a, record, november

n-gram Language Models

Can extend to trigrams, 4-grams, 5-grams, ...

In general this is an insufficient model of language

“The computer which I had just put into the machine room on the fifth floor crashed.”

Long-distance / Long-range dependencies

But we can often get away with n-gram models, where n is a small number

Estimating bigram probabilities

The maximum likelihood estimate

$$P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$



What happens when $i=1$?

Special edge case tokens: $\langle s \rangle$ and $\langle /s \rangle$ for beginning of sentence and end of sentence, respectively

An example

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(\text{I} | \text{<s>}) = \frac{2}{3} = .67$$

$$P(\text{Sam} | \text{<s>}) = \frac{1}{3} = .33$$

$$P(\text{am} | \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} | \text{Sam}) = \frac{1}{2} = 0.5$$

$$P(\text{Sam} | \text{am}) = \frac{1}{2} = .5$$

$$P(\text{do} | \text{I}) = \frac{1}{3} = .33$$

Berkeley Restaurant Corpus (BRP)

- can you tell me about any good cantonese restaurants close by
- mid priced thai food is what i'm looking for
- tell me about chez panisse
- can you give me a listing of the kinds of food that are available
- i'm looking for a good place to eat breakfast
- when is caffe venezia open during the day

Total: 9222 similar sentences

BRP: Raw Counts

Out of 9222 sentences

Unigrams

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

Bigrams

History

Next Word

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

BRP: Bigram Probabilities

$$P(w_i|w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

Bigram Probabilities: Raw bigram counts normalized by unigram counts

w_i

	i	want	to	eat	chinese	food	lunch	spend
w_{i-1}	i	0.002	0.33	0	0.0036	0	0	0.00079
	want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054
	to	0.00083	0	0.0017	0.28	0.00083	0	0.0025
	eat	0	0	0.0027	0	0.021	0.0027	0.056
	chinese	0.0063	0	0	0	0	0.52	0.0063
	food	0.014	0	0.014	0	0.00092	0.0037	0
	lunch	0.0059	0	0	0	0	0.0029	0
	spend	0.0036	0	0.0036	0	0	0	0

What kinds of knowledge?

$$P(\text{english} | \text{want}) = .0011$$

$$P(\text{chinese} | \text{want}) = .0065$$

$$P(\text{to} | \text{want}) = .66$$

$$P(\text{eat} | \text{to}) = .28$$

$$P(\text{food} | \text{to}) = 0$$

$$P(\text{want} | \text{spend}) = 0$$

$$P(i | \langle s \rangle) = .25$$

Bigram estimates of sentence probabilities

$P(<s> \text{ I want english food } </s>) =$

$P(\text{I} | <s>)$

$\times P(\text{want} | \text{I})$

$\times P(\text{english} | \text{want})$

$\times P(\text{food} | \text{english})$

$\times P(</s> | \text{food})$

$= .000031$

Quite low...

Underflow Issues

We do everything in log space

- Avoid underflow
- Adding is faster than multiplying

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

Lecture Outline

1. Announcements + Recap 
2. Probabilistic Language Models 
3. n-gram Language Models 
4. Evaluation and Perplexity
5. Generating from an n-gram Language Model
 - i. Zeroes
6. Smoothing

Evaluation and Perplexity

How good is a language model?

Does our language model prefer good sentences to bad ones?

- Key Idea: Assign higher probability to “real” or “frequently observed” sentences than “ungrammatical” or “rarely observed” sentences

In practice we don't explicitly need to do the latter!

We train parameters of our model on a training set.

We test the model's performance on data we haven't seen.

- A test set is an unseen dataset that is different from our training set, totally unused.
- An evaluation metric tells us how well our model does on the test set.

Intuition of Perplexity

The Shannon Game: How well can we predict the next word?

I always order pizza with cheese and ____

The 33rd President of the US was ____

I saw a ____

mushrooms 0.1

pepperoni 0.1

anchovies 0.01

....

fried rice 0.0001

....

and 1e-100



Unigrams are terrible at this game!

A better model of a text is one which assigns a higher probability to the word that actually occurs

Perplexity

The best language model is one that best predicts an unseen test set

- Gives the highest $P(\text{sentence})$ for most sentences acceptable to humans

$$PPL(\mathbf{w}) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}}$$

Perplexity is the inverse probability of the test set, normalized by the number of words

Minimizing perplexity is the same as maximizing probability

Chain rule:

Applying Markov's assumption for bigrams:

$$PPL(\mathbf{w}) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1, w_2, \dots, w_N)}}$$

$$= \sqrt[N]{\frac{1}{\prod_i P(w_i | w_1, \dots, w_{i-1})}}$$

$$= \sqrt[N]{\frac{1}{\prod_i P(w_i | w_{i-1})}}$$

Perplexity Example

Let's suppose a sentence of length 50 consisting of random digits

$$P(w) = \frac{1}{10}$$

What is the perplexity of this sentence according to a model that assigns uniform probability to each digit?

$$\begin{aligned} PPL(w) &= p(w_1, w_2, \dots, w_N)^{-\frac{1}{N}} \\ &= \left[\left(\frac{1}{10}\right)^{50}\right]^{-\frac{1}{50}} \\ &= 10 \end{aligned}$$

Lower perplexity = better model!

Training 38 million words, test 1.5 million words, from the Wall Street Journal

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109



What are the two things that might affect perplexity?

Lecture Outline

1. Announcements + Recap 
2. Probabilistic Language Models 
3. n-gram Language Models 
4. Evaluation and Perplexity 
5. Generating from an n-gram Language Model
 - i. Zeroes
6. Smoothing

Generating from an n-gram model and Zeros

Recall: BRP

$$P(\text{english} | \text{want}) = .0011$$

$$P(\text{chinese} | \text{want}) = .0065$$

$$P(\text{to} | \text{want}) = .66$$

$$P(\text{eat} | \text{to}) = .28$$

$$P(\text{food} | \text{to}) = 0$$

$$P(\text{want} | \text{spend}) = 0$$

$$P(i | \langle s \rangle) = .25$$



How can we generate sentences from this bigram model?

Generating from a bigram model

- Choose a random bigram ($\langle s \rangle$, w) according to its probability
- Now choose a random bigram (w , x) according to its probability
- And so on until we choose $\langle /s \rangle$
- Then string the words together

$\langle s \rangle$ I

I want

want to

to eat

eat Chinese

Chinese food

food $\langle /s \rangle$

I want to eat Chinese food

The WSJ is no Shakespeare!

1
gram

Months the my and issue of year foreign new exchange's september were recession exchange new endorsed a acquire to six executives

2
gram

Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor would seem to complete the major central planners one point five percent of U. S. E. has already old M. X. corporation of living on information such as more frequently fishing to keep her

3
gram

They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions

Shakespearean n-grams

1
gram

–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

–Hill he late speaks; or! a more to leg less first you enter

2
gram

–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.

–What means, sir. I confess she? then all sorts, he is trim, captain.

3
gram

–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

–This shall forbid it should be branded, if renown made it empty.

4
gram

–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

–It cannot be but so.

Shakespeare as a corpus



$N=884,647$ tokens, $V=29,066$

Shakespeare produced 300,000 bigram types out of $V^2=844$ million possible bigrams

So 99.96% of the possible bigrams were never seen (have zero entries in the table)

4-grams (quadrigrams) are rarer still...

What's coming out looks like Shakespeare because it is Shakespeare!

Most n-grams are never seen!



So why not just sample from very high order n-gram models? Do we even need GPT-style LLMs?

The successes we are seeing here is a phenomena commonly known as overfitting

Overfitting is bad!

n-grams only work well for word prediction if the test corpus looks like the training corpus

- In real life, it often doesn't
- We need to train robust models that generalize!
 - Technical terms for “doing well on the test data” or “doing well on any test data”
- One kind of generalization: Zeros!
 - Things that don't ever occur in the training set, but occur in the test set

Zeros

Training set:

... denied the allegations
... denied the reports
... denied the claims
... denied the request

Test set

... denied the offer
... denied the loan

$$P(\text{offer} | \text{denied the}) = 0$$



will assign 0 probability to the test set!

What happens to perplexity??

One solution: the UNK token

A token is a technical term in NLP for what is commonly referred to as a word

Problem: Word “offer” didn’t appear in the train set...many words like “Obfuscate” won’t appear in most training sets!

These are known as OOV for “out of vocabulary”, or unknown tokens

One way to handle OOV tokens is by adding a pseudo-word called <UNK>

We can replace all words that occur fewer than n times in the training set (where n is some small number) by <UNK> and re-estimate the counts and probabilities

When not done carefully, may artificially lower perplexity

Welcome

CSCI 544: Applied NLP

- TODOs for you
 - Start forming project teams – **due Sep 3 !**
- Next Class
 - n-gram Language Models (cont.) & logistic regression